There are restrictions we want to keep

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

A nested behaviour is spawned synchronously,
but can run independently of it’s parent

There are restrictions we want to keep

```
def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
} } }
```

When executed concurrently, the update behaviours and logging behaviours should agree on the order of updates.